## Internet of Things (IoT)

### CAPITOLO 1: Fondamenti, Visione e Sfide Architetturali

#### 1.1 L'Evoluzione delle Reti: Le 3 "E"

La trasformazione verso l'IoT si articola su tre vettori fondamentali:

- **Extensions (Estensioni)**: Espansione fisica della rete. Il paradigma evolve da *Any TIME, Any PLACE* (connettività ovunque e sempre per gli umani) a **Any THING** (connettività per gli oggetti). Si passa dalle reti *Human-to-Human* a *Machine-to-Machine (M2M)*. La rete si estende per includere miliardi di nodi fisici.
- **Expansions (Espansioni)**: Potenziamento della capacità trasmissiva. La rete necessita di banda larga (*Broadband*) e infrastrutture robuste per gestire il traffico immenso generato dai miliardi di sensori.
- **Enhancements (Miglioramenti)**: Aumento dell'intelligenza intrinseca della rete. Si sviluppano le *Smart Networks*, con approcci **Data-centric** (dove il contenuto del dato è più importante dell'indirizzo del nodo che lo ha generato) e sistemi **Context-aware** (capaci di comprendere il contesto fisico, es. posizione, temperatura, e adattare il proprio comportamento).

#### 1.2 Il Duplice Cambiamento di Scala: Scale UP e Scale DOWN

L'IoT si fonda su un paradosso infrastrutturale che ne determina le sfide tecnologiche:

- **Scale UP (Crescita macroscopica)**: Aumento esponenziale del numero di dispositivi. Si stima il dispiegamento di oltre 50 miliardi di smart objects. L'impatto sulla rete (indirizzamento, routing, data collection) è devastante, rendendo obsoleto l'IPv4.
- **Scale DOWN (Riduzione microscopica)**: Per permettere lo *Scale UP*, i singoli dispositivi devono essere ridotti all'osso:
  - **Costo**: Da centinaia di dollari a pochi centesimi per sensore, permettendo dispiegamenti massivi ("usa e getta").
  - **Risorse Hardware**: Le memorie crollano dai Gigabyte ai **KiloByte (KB)**. Le CPU passano dai Gigahertz a pochi **Megahertz (MHz)**.
  - **Consumi**: Dai Watt si scende ai **milliWatt (mW)** o microWatt, permettendo a una batteria di durare anni.

#### 1.3 Sistemi Embedded vs Smart Objects

- **Sistemi Embedded**: Sistemi integrati per compiti specifici (es. centralina iniezione auto). Hanno risorse limitate, ma la **comunicazione esterna non è il loro scopo primario**, bensì una funzione opzionale. Spesso usano sistemi operativi Real-Time (RTOS).
- **Smart Objects**: Condividono gli stessi vincoli hardware degli embedded, ma **la comunicazione è la loro ragion d'essere**. Senza rete, uno smart object perde il suo scopo.

#### 1.4 Radici Storiche: Ubiquitous Computing e WSN

- **Ubiquitous e Pervasive Computing**: Concetto introdotto da Mark Weiser (anni '90). La tecnologia deve "scomparire" nell'ambiente.
  - *Esempi storici*: Active Badges (AT&T, tracciamento dipendenti), AmbientROOM (MIT, 1996, interfacce ambientali), Wearable computing.
- **WSN (Wireless Sensor Networks)**: Nati nei primi anni 2000 (es. progetto *Smart Dust*). Reti create per monitorare grandezze fisiche.
- **Il salto IoT**: L'IoT eredita dalle WSN la gestione del risparmio energetico estremo, ma si differenzia integrando gli oggetti direttamente nella rete Internet IP e aggiungendo la capacità di **Attuazione** (non solo rilevare dati, ma intervenire sull'ambiente modificandolo).

#### 1.5 Le Sfide Architetturali: Silos vs Horizontal

- In passato, i sistemi M2M erano **Silos Verticali** (*Stove pipes*): reti chiuse, protocolli proprietari, hardware specifico per un'unica applicazione (es. sensori di parcheggio che non potevano parlare con la rete di illuminazione pubblica).
- L'IoT mira a un'**Architettura Orizzontale** basata su **IP (Internet Protocol)**. Si introduce un *Service Capability Layer (SCL)*, un middleware che espone API standardizzate (es. RESTful, CoAP) permettendo a molteplici applicazioni software di usare la stessa infrastruttura fisica di sensori.

---

### CAPITOLO 2: Hardware del Nodo, Energia e Sistemi Operativi

#### 2.1 Architettura Hardware: I 4 Blocchi

Ogni nodo IoT è composto da 4 sottomoduli:

1. **Unità di Calcolo (Microcontrollore - MCU)**:
   - "System on a Chip" che include Microprocessore, Memoria e interfacce (ADC, I2C, SPI). Costo < 10$.
   - **ROM/Flash**: Contiene il firmware (istruzioni).
   - **RAM**: Estremamente ridotta (pochi KB), serve per i dati dinamici e i buffer di rete. La scarsità di RAM è il limite principale per implementare i protocolli Internet completi (es. TCP).
2. **Modulo di Comunicazione (Radio Transceiver)**:
   - È il componente più critico. Il consumo della radio in funzione è spesso 10 volte superiore a quello della CPU attiva.
   - **Il problema dell'Idle Listening**: Rimanere in ascolto sul canale radio senza ricevere nulla consuma quasi quanto la trasmissione (es. Tx = 15 mW, Rx = 12 mW, Idle = 12 mW).
3. **Sensori e Attuatori**: Traduttori dal dominio analogico/fisico al dominio digitale.
4. **Fonte di Alimentazione (Power Supply)**:
   - **Batterie**: Spesso a bottone o stilo (es. Litio AA). Sostituirle in reti da migliaia di nodi è proibitivo, l'obiettivo è la durata pari alla vita utile dell'oggetto (5-10 anni).
   - **Power Scavenging (Energy Harvesting)**: Estrazione di energia dall'ambiente: celle fotovoltaiche, piezoelettrico (vibrazioni), generatori termoelettrici, RF scavenging (energia dalle onde radio).

### 2.2 Duty Cycling: La Strategia di Sopravvivenza

Per garantire anni di autonomia, il nodo deve rimanere in stato di **Sleep** (riposo profondo, radio spenta, consumi < 0.02 mW) per la stragrande maggioranza del tempo (es. 99.9%). Il **Duty Cycle** è il rapporto percentuale tra tempo di attività e tempo totale.
La sfida dei protocolli di rete (specialmente a livello MAC) è **sincronizzare** i momenti in cui i dispositivi si svegliano per permettere la comunicazione senza sprecare energia in lunghe attese (Idle Listening).

### 2.3 Sistemi Operativi IoT

A causa delle risorse limitate (niente Memoria Virtuale, MMU assente), gli OS per IoT sono minimalisti:

- **Contiki (ora Contiki-NG)**: Scritto in C. Storicamente fondamentale per aver portato l'IPv6 sui sensori (tramite gli stack uIP e uIPv6). Non usa veri thread ma **protothreads** (che condividono un unico stack per risparmiare RAM). Include il celebre emulatore di rete **Cooja**.
- **TinyOS**: Scritto nel linguaggio **nesC**. Architettura basata su componenti cablati staticamente a tempo di compilazione. Fortemente orientato agli eventi, non usa blocchi e thread tradizionali. Un pilastro storico per le WSN.
- **FreeRTOS**: Kernel Real-Time leggero, molto usato nell'industria. Supporta il multithreading classico, garantisce tempi di esecuzione deterministici, ma richiede leggermente più memoria degli altri.

---

## CAPITOLO 3: Stack di Rete e Livello Trasporto (TCP/UDP)

L'IoT abbraccia la suite TCP/IP superando i protocolli proprietari, permettendo ai sensori di essere veri e propri cittadini di Internet. Il livello di Trasporto usa il concetto di **Socket (Indirizzo IP + Numero di Porta a 16 bit)** per instradare i pacchetti ai corretti processi applicativi.

### 3.1 UDP (User Datagram Protocol)

- **Modello**: Connectionless (senza connessione), Best-effort (inaffidabile), Stateless (senza stato).
- **Header**: Minimo, esattamente **8 byte**. Campi: *Source Port (16 bit), Destination Port (16 bit), Length (16 bit), Checksum (16 bit)*. Nota: in IPv6 il Checksum UDP diventa obbligatorio perché rimosso dall'header IP.
- **Vantaggi per l'IoT**: Assenza di overhead da *handshake* (connessione immediata), non occupa RAM per mantenere lo stato della connessione, overhead di rete ridicolmente basso.
- **Svantaggi**: Se il pacchetto si perde, è perso. L'affidabilità deve essere garantita a livello applicativo (es. tramite protocollo CoAP).

### 3.2 TCP (Transmission Control Protocol)

- **Modello**: Connection-oriented (circuito virtuale), Reliable (affidabile), In-order (consegna in ordine).
- **Meccanismi di Affidabilità base**:
  1. **Acknowledgment (ACK)**: Il destinatario invia conferme. Il mittente ritrasmette se non riceve l'ACK entro un Timeout.
  2. **Numeri di Sequenza**: Ogni byte inviato ha un numero di sequenza. Risolve i problemi di duplicazione e disordine.
  3. **Sliding Window (Controllo di Flusso)**: Il destinatario invia nell'header la *Advertised Window* (o *Window Size*), indicando quanti byte può ancora accettare nel suo buffer. Evita il sovraccarico del destinatario.

### 3.3 Calcolo del Timeout TCP e RTT

Poiché le reti sono variabili, il timeout non può essere statico. Si calcola basandosi sul *Round Trip Time (RTT)*:

- **Calcolo Dinamico dell'RTT**:
  $EstimatedRTT = \alpha \cdot EstimatedRTT + (1 - \alpha) \cdot SampleRTT$
  Tipicamente $\alpha = 0.875$. È una media mobile ponderata che smorza le variazioni improvvise.
- **Timeout**:
  $TimeOut = 2 \cdot EstimatedRTT$
- **Fast Retransmit**: Se arrivano 3 ACK identici (duplicati), significa che un pacchetto intermedio si è perso ma i successivi stanno arrivando. Il mittente ritrasmette immediatamente senza aspettare il TimeOut. (Inutile nell'IoT perché i flussi dati sono troppo brevi per generare 3 ACK successivi).

### 3.4 Stabilimento della Connessione TCP (Three-Way Handshake)

Necessario per concordare gli **ISN (Initial Sequence Number)**:

1. Host A invia un segmento **SYN** con numero di sequenza $ISN_A$.
2. Host B risponde con un segmento **SYN-ACK**, contenente il suo $ISN_B$ e un ACK di approvazione pari a $ISN_A + 1$.
3. Host A conferma inviando un **ACK** pari a $ISN_B + 1$.

#### Il Problema dei Pacchetti Fantasma e l'Orologio ISN

Se una connessione si chiude e se ne riapre subito una identica sulle stesse porte, i pacchetti ritardati della vecchia connessione ("pacchetti fantasma") potrebbero essere accettati come validi se il numero di sequenza partisse sempre da 0.

- **Soluzione**: L'ISN viene generato da un contatore (orologio) a 32 bit che si incrementa di 1 ogni **4 microsecondi**.
- Il contatore esegue un "wrap-around" (ritorna a zero) in circa **4.55 ore** ($2^{32} \times 4\mu s$). Questo garantisce che nessun pacchetto sopravviva in rete per 4.55 ore, rendendo impossibile la collisione tra vecchie e nuove connessioni.

### 3.5 Il Problema del TCP nell'IoT

Implementare il TCP completo in un nodo con pochi KB di RAM è proibitivo a causa dei grossi buffer necessari per la *Sliding Window*.
**Soluzione (TCP ridotto)**: Negli stack IoT minimali, la Sliding Window viene eliminata. La dimensione della finestra viene posta pari a un solo MSS (Maximum Segment Size). Il TCP si riduce a un comportamento **"Stop-and-Wait"**: invio 1 segmento, mi fermo, aspetto l'ACK, invio il successivo. Il throughput crolla, ma la RAM viene risparmiata.

---

## CAPITOLO 4: IPv6, Indirizzamento e ICMPv6

### 4.1 L'Esaurimento di IPv4 e i Limiti del NAT

L'IPv4 (32 bit = 4.3 miliardi di indirizzi) è esaurito. Il NAT (Network Address Translation) è stato introdotto come soluzione temporanea per condividere un IP pubblico tra più nodi privati.

- **I danni del NAT nell'IoT**: Rompe il paradigma "End-to-End". Rende i dispositivi privati non direttamente raggiungibili dall'esterno (problema per gli attuatori IoT). Interferisce pesantemente con protocolli di sicurezza come IPSec, poiché modifica gli indirizzi IP e ricalcola i checksum in transito, invalidando le firme di integrità dell'Autentication Header (AH).

### 4.2 L'Architettura IPv6 (RFC 2460)

Costruito nativamente per scalabilità, sicurezza ed efficienza di elaborazione nei router.

- **Spazio di Indirizzamento**: **128 bit**. Permette $3.4 \times 10^{38}$ indirizzi.
- **Rappresentazione**: 8 blocchi da 16 bit in esadecimale separati da due punti (es. `2001:0db8:85a3:0000:0000:8a2e:0370:7334`).
  - *Regole di compressione*: Omissione degli zeri iniziali in ogni blocco. Sequenze di blocchi a zero possono essere collassate con `::` (usabile una sola volta per indirizzo).
- **Rimozione del Broadcast**: Non esiste più il broadcast puro, tutto è gestito elegantemente tramite **Multicast** o **Anycast**.

### 4.3 Tipologie e Gerarchia di Indirizzi IPv6

L'indirizzo è logicamente diviso in Prefisso di rete e **Interface ID (solitamente 64 bit)**.

1. **Unspecified (`::/128`)**: Nessun indirizzo, usato solo come sorgente temporanea in fase di startup.
2. **Loopback (`::1/128`)**: Equivalente a 127.0.0.1.
3. **Link-Local Unicast (`FE80::/10`)**: Indirizzo fondamentale e **obbligatorio** su ogni interfaccia IPv6. Viene usato per la comunicazione esclusiva all'interno dello stesso segmento fisico/radio. I router non inoltrano mai pacchetti con questi indirizzi.
4. **Global Unicast (`2000::/3`)**: L'indirizzo pubblico instradabile su Internet. Strutturato in:
   - *Global Routing Prefix* (tipicamente 48 bit, assegnato dall'ISP).
   - *Subnet ID* (tipicamente 16 bit, definito dall'amministratore locale).
   - *Interface ID* (64 bit, identifica univocamente l'host sulla subnet).
5. **Multicast (`FF00::/8`)**: Identifica un gruppo di interfacce.
6. **Anycast**: Indirizzo assegnato a più router/server, il routing porta il pacchetto al nodo Anycast topologicamente più vicino.

#### Derivazione Modified EUI-64

Meccanismo per generare l'Interface ID di 64 bit a partire da un MAC address di 48 bit (tipico di Ethernet/Wi-Fi):

1. Si taglia il MAC a metà.
2. Si inseriscono i 16 bit speciali `FFFE` nel centro.
3. Si **inverte il 7° bit** del MAC originale (Universal/Local bit).

### 4.4 Autoconfigurazione (SLAAC)

IPv6 permette lo **Stateless Address Autoconfiguration**. Il nodo accende l'interfaccia, si auto-assegna un indirizzo Link-Local, ascolta i Router Advertisement (messaggi ICMPv6 dal router locale), prende il Prefisso di Rete Globale diffuso dal router e ci appiccica il proprio Interface ID (es. generato via EUI-64). Nessun server DHCP è richiesto.

### 4.5 L'Header IPv6 Ottimizzato

L'header è a lunghezza fissa di **40 byte** ed è allineato a 64 bit, permettendo ai router di processarlo fulmineamente in hardware.
**Campi dell'Header Base (40 byte)**:

1. **Version** (4 bit): Sempre valore 6.
2. **Traffic Class** (8 bit): Qualità del servizio (QoS), sostituisce il ToS IPv4.
3. **Flow Label** (20 bit): Mantiene traccia di flussi di pacchetti (es. streaming audio/video in real-time) che richiedono lo stesso instradamento o gestione prioritaria nei router intermedi.
4. **Payload Length** (16 bit): La lunghezza del pacchetto esclusi i 40 byte dell'header base. Max 64 KB (salvo uso Jumbo Payload).
5. **Next Header** (8 bit): Identifica il protocollo successivo (es. TCP, UDP) o, innovazione di IPv6, il primo **Extension Header**.
6. **Hop Limit** (8 bit): Sostituisce il TTL. Viene decrementato da ogni router. Se 0, pacchetto scartato.
7. **Source Address** (128 bit).
8. **Destination Address** (128 bit).

**Differenze critiche da IPv4**:

- **Nessun Checksum IP**: Il controllo di integrità dell'header è rimosso. Si presume che il Layer 2 (MAC) e il Layer 4 (UDP/TCP, che ora ha un checksum obbligatorio e calcolato su uno pseudo-header IP) garantiscano l'integrità. I router non devono ricalcolare il checksum ad ogni Hop, risparmiando tempo.
- **Nessuna Frammentazione Intermedia**: In IPv6, **i router intermedi non frammentano mai i pacchetti**. Se un pacchetto eccede la MTU del link uscente, il router lo scarta e manda un errore ICMPv6 "Packet Too Big" al mittente. Spetta al mittente scoprire la Path MTU e frammentare l'invio alla fonte.

#### 4.6 Gli Extension Headers (EH)

Tutte le funzionalità opzionali sono state tolte dall'header base e messe in "vagoni" chiamati Extension Headers. Si legano "a margherita" usando il campo *Next Header*.

- Devono apparire in un ordine preciso se presenti.
- Devono avere una lunghezza multipla di 8 byte.
- **L'ordine obbligatorio**:
  1. **Hop-by-Hop Options**: L'unico letto ed esaminato da ogni singolo router lungo il percorso. Contiene il *Jumbo Payload option* per payload > 65535 byte. Strutturato in TLV (Type-Length-Value) con byte di padding ( `Pad1`, `PadN`) per mantenere l'allineamento a 64 bit.
  2. **Destination Options** (pre-routing).
  3. **Routing Header**: Permette al mittente di dettare il percorso. Contiene una lista di indirizzi da visitare. Il campo `Segments Left` indica quanti hop mancano. Il nodo raggiunto scambia il proprio indirizzo con il successivo nella lista.
  4. **Fragment Header**: Gestisce la frammentazione lato mittente. Lungo 8 byte fisse. Il pacchetto originale è diviso in *Un-fragmentable part* (che deve viaggiare intera in ogni frammento, es. header base e routing) e *Fragmentable part*.
  5. **Authentication Header (AH) / ESP**: Protocolli di sicurezza IPSec.
  6. **Destination Options** (post-routing): Lette solo dalla destinazione finale.

#### 4.7 ICMPv6 e Neighbor Discovery

ICMP in IPv6 diventa il perno operativo della rete, assorbendo funzioni prima svolte dall'ARP o dall'IGMP.
**Funzioni ICMPv6**: Messaggi di errore (Destination Unreachable, Packet Too Big, Time Exceeded) e messaggi informativi (Echo Request/Reply per il PING).
**Neighbor Discovery Protocol (NDP)**: Basato su ICMPv6, risolve gli indirizzi IP in MAC address (sostituisce l'ARP dell'IPv4 che usava il broadcast).

- **Neighbor Solicitation**: "Chi possiede questo IPv6?" inviato al gruppo Multicast di quell'indirizzo. Usato anche per la DAD (Duplicate Address Detection) all'avvio.
- **Neighbor Advertisement**: La risposta unicast "Sono io, ecco il mio MAC".

---

### CAPITOLO 5: 6LoWPAN e l'Adattamento per le LLN

#### 5.1 Il Problema: IPv6 sopra 802.15.4

IPv6 è stato progettato per reti cablate (Ethernet), imponendo una MTU minima garantita dal livello di collegamento di **1280 byte**. Tuttavia, le reti IoT a basso consumo (Low-power and Lossy Networks - LLN) utilizzano spesso lo standard radio IEEE 802.15.4, che offre frame microscopici.

- **La MTU del 802.15.4 è di soli 127 byte**.
- **Il calcolo del Worst-Case Scenario**:
  - Dagli 127 byte togliamo l'header MAC completo (25 byte).
  - Togliamo l'overhead massimo di sicurezza a livello Link (AES-128 con MIC, 21 byte).
  - Restano **81 byte** effettivi per trasportare l'intero pacchetto IPv6.
  - Se consideriamo l'header IPv6 standard (40 byte) e l'header UDP (8 byte), lo spazio rimanente per i dati utili (Payload applicativo) si riduce alla miseria di **33 byte!** (E con TCP di 20 byte, ne restano **21**).
- Questa proporzione inaccettabile tra overhead e payload rende essenziale un livello intermedio: il **6LoWPAN Adaptation Layer**.

#### 5.2 Architettura delle Reti 6LoWPAN

Le reti 6LoWPAN sono generalmente organizzate come *Stub Networks* (reti foglia) attaccate a Internet tramite un Gateway o Border Router (LBR).

- **Simple LoWPAN**: Un singolo Edge Router.
- **Extended LoWPAN**: Più Edge Router uniti da una dorsale comune.
- **Ad-Hoc LoWPAN**: Rete isolata senza collegamento a Internet.

#### 5.3 Il 6LoWPAN Adaptation Layer

Collocato tra il MAC (802.15.4) e l'IP (IPv6), sfrutta la tecnica dell'**Header Stacking** (impilamento di sub-header) identificati da un byte iniziale (*Dispatch byte*). Fornisce tre servizi principali:

##### 5.3.1 Frammentazione e Riassemblaggio

Quando l'IPv6 genera pacchetti superiori agli ~80 byte utili, il 6LoWPAN interviene frammentando:

- **Fragment Header**:
  - Il primo frammento contiene `Datagram_size` (11 bit, la dimensione totale del datagramma IP originale in unità di 8 byte) e `Datagram_tag` (16 bit, identifica il pacchetto).
  - I frammenti successivi contengono le stesse informazioni e aggiungono un `Datagram_offset` (8 bit) per indicare la posizione del frammento (sempre in multipli di 8 byte).
- **Vulnerabilità**: Se un solo microscopico frammento Link-Layer va perduto (cosa frequente in radio), non c'è ritrasmissione locale: l'intero datagramma IPv6 frammentato risulterà incompleto e il destinatario lo scarterà interamente!

#### 5.3.2 Header Compression (HC1, HC2 e IPHC)

La compressione in 6LoWPAN è *Flow-independent* (non basata sullo storico delle comunicazioni passate come l'ROHC), ma si basa sulle dipendenze tra i livelli (*cross-layer*) e sui valori fissi.

- **I 3 Principi Base**:
  1. Derivazione: l'Interface ID dell'IPv6 si calcola direttamente dall'indirizzo MAC (64-bit EUI) del layer 802.15.4.
  2. Soppressione dei valori ovvi (Versione=6, Lunghezza dedotta dal frame radio, Flow Label/TC spesso a 0).
  3. Contesti Condivisi: Prefissi di rete noti a priori.
- **HC1 (Header Compression 1)**: Progettato per gli indirizzi *Link-Local*. In caso perfetto (comunicazione locale pura), l'header da 40 byte crolla a **3 byte**:
  - `Dispatch Byte` (es. `01000010` = HC1 in arrivo).
  - `HC1 Byte` (bitmask che indica quali campi IP sono stati tagliati).
  - `Hop Limit` (l'unico campo originale IP che viene sempre inviato intero, 1 byte).
- **HC2 (UDP Compression)**: Impilato dopo HC1. Permette di comprimere le porte UDP (16 bit) in un range ridotto. Tramite l'impostazione dei bit dell'HC2, il mittente invia solo pochi bit per la porta (*short-value*); il destinatario somma allo *short-value* il valore fisso **61616** per recuperare la porta originaria. Questo "trucco" riduce l'header UDP da 8 byte a **3 byte** (1 per le porte, 2 per l'intoccabile checksum).
- **IPHC (ImProved Header Compression)**: Supera HC1 perché capace di comprimere indirizzi IPv6 Globali e Multicast (non solo Link-Local). Usa il concetto del **CID (Context Identifier)** (1 byte addizionale diviso a metà, 4 bit SCI per il source context e 4 bit DCI per il destination context). Permette di mappare fino a 16 prefissi di rete (contesti) concordati. Nel caso migliore (usando il contesto di default 0), un colossale pacchetto IPv6 Globale+UDP può avere l'intestazione schiacciata a soli **4 byte** (o addirittura **2 byte** se si decide, rischiando, di sacrificare il Checksum UDP con il *NHC*).

#### 5.3.3 Link-Layer Forwarding e Mesh Header

Il routing in una WSN può avvenire a due livelli:

- **Route-Over**: Il routing avviene a livello IP (Layer 3). Ogni salto radio è considerato un nuovo hop IP, e l'Hop Limit IPv6 viene scalato a ogni nodo.
- **Mesh-Under**: Il routing avviene "sotto" l'IP, a livello Link (Layer 2), trasparente al layer di rete. L'intero cluster di sensori appare all'IPv6 come un'unica gigantesca sottorete broadcast.
  - Usa il **Mesh Header** 6LoWPAN. Contiene gli indirizzi estremi (Originator IP e Final Destination IP), mentre gli indirizzi MAC nel frame fisico cambiano a ogni salto radio. Aggiunge un campo `Hop Left` a livello L2 per tranciare pacchetti in loop. Attuabile solo dai dispositivi completi (FFD - Full Function Devices).

---

### CAPITOLO 6: Accesso al Mezzo Radio (MAC) e Livello Fisico

#### 6.1 Pattern di Comunicazione WSN

1. **Uno-a-Uno (1:1)**: Comunicazioni Point-to-Point (P2P). Per comunicare oltre il raggio fisico, il nodo si appoggia ad altri nodi (relay multi-hop). La potenza decresce drasticamente: Potenza Ricevuta $P_r = \frac{P_t G_t G_r}{c \cdot d^\beta}$ (dove $d$ è la distanza e $\beta$ è l'esponente di caduta, compreso tra 2 e 4). Invece di inviare un segnale potentissimo da A a C, energeticamente conviene molto di più usare un nodo B intermedio, regolando la potenza Tx pacchetto per pacchetto in base all'RSSI del destinatario.
2. **Uno-a-Molti (1:M)**: Gateway verso nodi. Usa il *Flooding* (diffusione epidemica). Ottimizzazioni tramite *Gossiping* e protocollo algoritmico *Trickle* per evitare tempeste di traffico (Broadcast storm).
3. **Molti-a-Uno (M:1)**: Pattern principe dell'IoT (Data Collection verso il Sink). Richiede routing ad albero robusto.

#### 6.2 Principi di Accesso Multiplo (Multiplexing)

Per far parlare più nodi sullo stesso canale senza interferire:

- **SDMA (Space)**: Celle geograficamente separate o antenne direzionali.
- **FDMA (Frequency)**: Canali radio separati o Frequency Hopping (es. FHSS del Bluetooth, si cambia canale 1600 volte al secondo).
- **TDMA (Time)**: Turni di tempo (scheduling). Ottimo per evitare collisioni, pessimo per flessibilità (chi è zitto spreca il turno).
- **CDMA (Code)**: Nodi trasmettono sulla stessa frequenza nello stesso momento ma usano "codici matematici" ortogonali. Il ricevitore fa il prodotto scalare per estrarre la singola trasmissione. Soffre gravemente del *Near-Far effect* (se il vicino urla, copre il lontano), richiede rigoroso controllo automatico della potenza (Power Control).

#### 6.3 I Problemi Intrinseci delle Comunicazioni Wireless

L'ambiente radio è un mezzo condiviso (broadcast) e inaffidabile:

1. **Il problema del Terminale Nascosto**: A e C non si percepiscono perché troppo lontani (fuori dal rispettivo Carrier Sense range). Entrambi pensano che il canale sia libero e trasmettono simultaneamente a B (che è nel mezzo). I pacchetti si scontrano in B e vanno persi.
   - *Soluzione Storica*: Scambio **RTS/CTS** (Request To Send / Clear To Send). A manda un breve RTS a B. B risponde con un CTS "a tutto volume". C, che non sentiva A ma sente il CTS di B, capisce che B è occupato in ricezione e tace. Inserisce il concetto di *Virtual Carrier Sensing* tramite il NAV (Network Allocation Vector): dentro l'RTS/CTS c'è scritto l'esatto tempo in cui il canale sarà occupato.
2. **Il problema del Terminale Esposto**: C vuole trasmettere a D. B sta trasmettendo ad A. C ascolta il canale (CSMA), sente B, e quindi per educazione decide di attendere all'infinito. In realtà, le due trasmissioni (B->A e C->D) non si disturberebbero affatto perché le destinazioni sono opposte e lontane!

#### 6.4 Protocolli MAC Basati su Contesa (Aloha e CSMA)

- **Slotted Aloha**: Il tempo è diviso in slot. Se due parlano nello stesso slot, collidono, attendono un numero casuale di slot e riprovano. Efficienza massima teorica: $1/e$ (circa **36%**).
  - *Modello Matematico*: Probabilità successo $P_1 = p(1-p)^{n-1}$.
- **Pure Aloha**: Si trasmette a caso. Efficienza dimezzata: $1/2e$ (circa **18%**).
- **Adaptive Slotted Aloha**: La rete stima il numero di nodi attivi $\hat{n}$ e imposta dinamicamente la probabilità ottimale di trasmissione a $p = 1/\hat{n}$. Se si sentono troppi silenzi, $p$ aumenta. Se si sentono troppe collisioni, $p$ diminuisce.

- **CSMA (Carrier Sense Multiple Access)**: "Ascolta prima di parlare".
  - *Non-persistente*: Rileva canale occupato, backoff, torna a dormire, riprova.
  - *p-persistente*: Rileva canale occupato, continua ossessivamente ad ascoltare, appena libero trasmette con probabilità $p$.
- **Binary Exponential Backoff**: Dopo ogni collisione (mancato ACK per $k$ tentativi), l'attesa randomica viene scelta in una finestra che raddoppia ($[1 \dots 2^k]$) oppure si riduce la probabilità di trasmissione $p = 2^{-k}$. Problema: per carichi di traffico oltre la soglia critica ($\lambda > 0.568$), diventa instabile causando un blocco totale della rete. Il *Polynomial Backoff* risolve crescendo più linearmente (stabile per $\lambda < 1$).

### 6.5 Standard Fisici per IoT

#### IEEE 802.15.4 (Base per ZigBee, Thread, 6LoWPAN)

Pensato specificamente per Low-Rate WPAN (Personal Area Networks).

- **Livello Fisico (PHY)**: Modulazione DSSS (Direct Sequence Spread Spectrum).
  - Frequenze: 868 MHz (Europa, 1 canale, 20 kb/s), 915 MHz (USA, 10 canali, 40 kb/s), **2.4 GHz (Globale, 16 canali, 250 kb/s)**.
  - I canali 2.4GHz del 15.4 si sovrappongono rovinosamente ai canali del Wi-Fi 802.11. L'unica salvezza sono i **canali 25 e 26** del 15.4 che si posizionano nello spazio libero agli estremi delle frequenze Wi-Fi.
  - Potenza massima consentita per non aver bisogno di licenze: **1 mW**.
- **Livello MAC**:
  - Due tipi di nodi: FFD (Full Function Device, fungono da router/coordinator) e RFD (Reduced Function Device, solo foglie terminali).
  - **Modalità Beaconless**: Rete destrutturata ad accesso CSMA/CA asincrono. Semplice ma meno efficiente energeticamente.
  - **Modalità Beacon-enabled**: Gestita a turni strutturati tramite i **Superframe**. Il PAN Coordinator invia periodicamente un *Beacon*. Il Superframe è diviso in:
    1. *Contention Access Period (CAP)*: I nodi lottano col CSMA/CA per trasmettere best-effort.
    2. *Contention Free Period (CFP)*: Il coordinatore alloca Guaranteed Time Slots (GTS) per i nodi che necessitano latenza e banda garantita (TDMA).
    3. *Inactive Period*: Tutti possono dormire, spegnendo la radio.

#### Wi-Fi (IEEE 802.11) nell'IoT

Il Wi-Fi tradizionale (802.11b/g/n) è letale per le batterie IoT a causa degli enormi assorbimenti (es. trasmette a 800 mW, resta in ascolto a 13 mW).
Tuttavia, nuove soluzioni hardware **Low-Power Wi-Fi** lo hanno reso viabile per applicazioni non estreme (es. prese smart casalinghe). Ottimizzano i circuiti analogici permettendo uno Sleep Mode a 0.2 mW e un *Standby profondo a soli* **0.018 mW**. Inoltre, sfruttano protocolli di wake-up rapidissimo, inviando raffiche di piccoli dati per poi ricadere nel deep sleep. Sfruttando la massiva infrastruttura Access Point preesistente, evitano di dover piazzare gateway aggiuntivi.

### 6.6 MAC Protocolli WSN (Focalizzati sul Risparmio Energetico)

L'obiettivo dei protocolli MAC per le Wireless Sensor Networks è uno solo: minimizzare l'energia, anche a costo di distruggere il throughput o alzare la latenza a livelli inaccettabili per reti tradizionali.

#### Il Problema Energetico e Modello di Consumo B-MAC

Le 4 cause di spreco in WSN:

1. **Collisioni**: Sprecano il costo della trasmissione (che andrà ripetuta).
2. **Overhearing**: Ascoltare passivamente un pacchetto indirizzato a un vicino.
3. **Control Overhead**: I byte e pacchetti spesi per dire "sono pronto", "ricevuto", "mi unisco".
4. **Idle Listening**: Tenere la radio accesa "nel dubbio che arrivi qualcosa" (il peggiore in assoluto).
   Il modello di Lifetime stimato per un nodo (es. Mica2) è:
   $E_{totale} = E_{ricezione} + E_{trasmissione} + E_{idle_listening} + E_{sensore} + E_{sleep}$

#### Protocolli Asincroni (Unscheduled)

- **B-MAC (Berkeley MAC)**: L'innovatore assoluto (2004). Introduce il concetto di **LPL (Low Power Listening)** o "Clear Channel Assessment (CCA) periodico". I nodi dormono sempre. Si svegliano una frazione di millisecondo, fanno un CCA (misurano l'energia nell'etere), se vuoto tornano a dormire.
  - *La trasmissione*: Come fa il mittente a beccare il destinatario sveglio? Il mittente prima dei dati invia un **Preambolo Continuo Lunghissimo** (più lungo del periodo di sleep del ricevitore).
  - *Difetto critico*: Genera un gravissimo **Overhearing**. Tutti i nodi nel vicinato, svegliandosi per il loro CCA periodico, sentono il lunghissimo preambolo, lo scambiano per attività, restano svegli inutilmente per tutto il tempo sprecando enormi quantità di batteria, solo per scoprire alla fine che il pacchetto dati era per qualcun altro. Il costo energetico è pesantemente spostato asimmetricamente sul mittente.
- **WiseMAC**: Risolve l'asimmetria del B-MAC. Impara i programmi di risveglio (*wake-up schedules*) dei vicini scambiando l'orario nell'ACK. Quando deve inviare, aspetta e inizia il preambolo B-MAC solo una frazione di millisecondo prima del risveglio programmato del ricevitore.
- **X-MAC (2006)**: Risolve l'Overhearing. Sostituisce il gigantesco preambolo continuo del B-MAC con una serie intermittente di **Micro-preamboli "a strobo"**, separati da brevi pause.
  - Nel micro-preambolo c'è scritto l'indirizzo del destinatario!
  - Un nodo si sveglia, legge l'indirizzo; se non è il suo, torna subito a dormire (Overhearing sconfitto).
  - Se è il suo, approfitta del gap tra due micro-preamboli per sparare un **Early ACK**. Il mittente riceve l'ACK, stoppa istantaneamente il ciclo di preamboli e invia subito il payload dati. Questo taglia di netto latenza e consumi del mittente.
- **ContikiMAC (2011)**: Integrato nel sistema operativo Contiki. Approccio rivoluzionario: abolisce del tutto i preamboli.
  - *Trasmissione*: Il mittente invia **ripetutamente lo stesso identico pacchetto dati completo** "a trenino", senza pause.
  - *Ricezione e vincoli temporali precisi*: Il ricevitore esegue due check CCA rapidissimi distanziati temporalmente in modo millimetrico. Il pacchetto trasmesso deve essere lungo abbastanza da non "cadere in mezzo" tra i due CCA, garantendo che almeno uno dei due check rilevi attività. Se rileva energia, il nodo resta sveglio per agganciare il successivo pacchetto del treno. Risponde con un ACK a livello Link e il mittente interrompe la ripetizione.
  - Include l'ottimizzazione del *Phase Lock*, imparando gli orari dei vicini proprio come WiseMAC, rendendo la trasmissione istantanea.

#### Protocolli Sincroni (Scheduled / TDMA)

- **S-MAC (Sensor MAC)**: Combatte tutte e 4 le piaghe energetiche imponendo la rigidità dei turni. I nodi si accordano su finestre temporali rigide composte da un periodo di *Listen* e uno di *Sleep*.
  - Sincronizzazione: I nodi si scambiano pacchetti SYNC per allineare gli orologi formando "vicinati virtuali".
  - Prevenzione collisioni: Durante la fase Listen attiva, chi vuole parlare prenota il canale usando lo scambio **RTS/CTS**.
  - Overhearing: Se un nodo sente un RTS o CTS non indirizzato a lui, usa l'informazione del NAV per calcolare quanto durerà lo scambio e spegne brutalmente la radio per l'intera durata della transazione (dormita selettiva).
  - *Difetto critico*: La rigidità del ciclo fisso (es. Listen 10%, Sleep 90%) non si adatta ai picchi di traffico o al silenzio totale, generando latenze inaccettabili in condizioni di carico, e soprattutto il problema dei **Nodi di Confine** (Border Nodes). Se una rete grande si frammenta in due gruppi con schedulazioni sfalsate, il nodo che si trova in mezzo deve per forza restare sveglio seguendo le tabelle orarie di entrambi i gruppi per fare da ponte (bridge), il che significa consumare il doppio della batteria esaurendosi prematuramente, tranciando a metà la rete.
- **T-MAC**: Miglioramento di S-MAC. Rende dinamico l'orario di spegnimento della finestra attiva in base alla misurazione del traffico reale (se c'è silenzio, stacca prima la finestra di listen).
- **Z-MAC (Hybrid)**: Tenta l'ibrido perfetto (Cross-layer). Sotto basso traffico si comporta come CSMA/CA (contesa casuale veloce). Quando il traffico si fa denso e l'etere saturo di collisioni, *shifta* autonomamente verso un comportamento puramente a turni TDMA, distribuendo equamente gli slot per gestire la congestione.

---

## CAPITOLO 7: Le Reti Geografiche LPWAN e LoRaWAN

Le tecnologie LPWAN (Low Power Wide Area Network) si posizionano dove le tecnologie cellulari tradizionali (2G/3G/LTE) falliscono per l'IoT: colmano il vuoto fornendo una connessione a raggio chilometrico (es. Smart City) con dispositivi a batteria dalla vita ultradecennale, al costo di velocità ridicolmente basse. Il vecchio cellulare legacy (2G/3G) non è più una base IoT affidabile a causa degli alti consumi, costi di licenza e della massiccia e definitiva dismissione globale delle reti.
*Paragone*: SigFox (modello di business totalmente chiuso, rete venduta "as a service", max 140 messaggi in uplink al giorno da 12 byte) vs **LoRaWAN** (specifica aperta e decentralizzata per il MAC, ma chip fisici brevettati Semtech, permette reti private o pubbliche).

### 7.1 LoRa (Il Livello Fisico / PHY)

Sviluppato e brevettato da Semtech. Consente connessioni di decine di chilometri senza bisogno di amplificatori.

- **Modulazione CSS (Chirp Spread Spectrum)**: Tecnica radaristica applicata ai dati. I bit vengono spalmati su frequenze che scivolano continuamente verso l'alto (Up-chirp) o verso il basso (Down-chirp). Questo garantisce un'immunità spaventosa al rumore, alle interferenze in banda stretta (es. Wi-Fi) e allo spostamento Doppler.
- **Link Budget Estremo**: LoRa raggiunge i **151 dB** di Link budget (contro i miseri 95 dB del Wi-Fi e i 121 dB del sub-GHz 6LoWPAN). La sensibilità del ricevitore arriva a -137 dBm, permettendo di demodulare segnali sepolti ben al di sotto della soglia di rumore di fondo termico!
- **Lo Spreading Factor (SF)**: Il parametro chiave che il progettista/server può manipolare (va da **SF7 a SF12**).
  - Un SF più alto allarga e dilata il tempo in cui un simbolo (gruppo di bit) viene mandato in aria, riducendo drasticamente il Data Rate ma incrementando massivamente la resistenza al rumore (range).
  - **Legge del +1**: Incrementare l'SF di +1 significa dimezzare il Bit Rate e raddoppiare l'Airtime (il tempo che la radio passa accesa consumando preziosa batteria), ma dona circa +2.5 dB di sensibilità in più.
  - **Relazioni Matematiche e Formule Fisiche LoRa**:
    - Numero di valori per simbolo = $2^{SF}$
    - Durata in tempo del singolo simbolo $D = \frac{2^{SF}}{B}$ (dove $B$ è la Bandwidth del canale, tipicamente 125 kHz).
    - Frequenza dei Simboli (Symbol Rate) $R_s = \frac{B}{2^{SF}}$
    - Bit Rate finale (Velocità netta) $R_b = SF \cdot R_s = SF \cdot \frac{B}{2^{SF}}$
  - **Ortogonalità**: Magia del CSS: due trasmissioni simultanee sulla *stessa esatta frequenza* ma con due *SF diversi* (es. SF7 e SF10) sono ortogonali; il ricevitore le tratta come due comunicazioni distinte e le decodifica entrambe senza che l'una corrompa l'altra.
- **Struttura del Frame LoRa Fisico**:
  - *Preambolo*: Fondamentale! Serve a calcolare il $\Delta t$ (sincronizzazione temporale), l'offset della portante $f_0$ e ad agganciare il *Symbol rate*.
  - *Header PHY e Payload*. I dati grezzi passano attraverso passaggi di indurimento: *Gray coding* (minimizza impatto di un errore di frequenza), *Whitening* (sbiancamento, evita lunghe sequenze di '0' o '1' di fila), *Interleaving* (sparpaglia i bit per resistere a disturbi isolati), e *FEC (Forward Error Correction)*.
- **Demodulazione FFT**: Il gateway moltiplica il segnale ricevuto per un "Down-chirp" noto (de-chirping), appiattendo l'onda in frequenze fisse pure. Applica poi una Trasformata di Fourier (FFT). Il picco massimo nella FFT indica con precisione assoluta il simbolo ricevuto.

### 7.2 LoRaWAN (Il Livello MAC e Rete)

Specifica gestita dalla LoRa Alliance. Sfrutta le frequenze libere ISM (In Europa tipicamente a **868 MHz**).

- **L'Architettura Network**: Modello "Stella di stelle" centralizzato.
  - **Gateway**: Stazioni base "stupide" (ponti radio trasparenti) che demodulano passivamente i segnali RF e li sputano su una dorsale IP verso il backend. Una trasmissione di un nodo (Uplink) non è diretta a un gateway specifico: viene semplicemente urlata nell'etere e captata da tutti i gateway nel raggio.
  - **Network Server (NS)**: Il vero intelligenze (cervello) della rete. Scarta i pacchetti duplicati captati da molteplici gateway, gestisce le conferme (ACK downlink), l'attivazione dei nodi, e ordina l'ADR.
  - **Application Server (AS)**: La fine della corsa per i dati grezzi. Ha le chiavi per decrittografare il payload utile.

#### 7.2.1 Regolamentazione: Accesso ALOHA e Duty Cycle

Non esiste Carrier Sensing. LoRaWAN usa l'accesso casuale puro **ALOHA**. Quando un sensore è pronto, accende la radio e urla (nessun "listen before talk"). Data la debolezza dell'ALOHA alle collisioni, la stabilità è salvata in Europa dall'imposizione legislativa sul **Duty Cycle dell'1%**. Per ogni secondo di tempo in cui un nodo occupa il canale trasmettendo, è legalmente obbligato a tacere per i successivi 99 secondi. Questo garantisce canali radio scarichi. Il payload ammissibile è minuscolo: dai **59 byte (SF12) fino a 230 byte (SF7)**.

#### 7.2.2 Le Classi dei Dispositivi LoRaWAN

1. **Classe A (All / Bidirezionale asincrono)**: Implementata obbligatoriamente da tutti i nodi. Massimizza l'efficienza energetica. Il nodo si sveglia, lancia il suo Uplink, poi apre obbligatoriamente **solo due minuscole finestre temporali di ricezione (RX1 e RX2)**, rispettivamente a +1s e +2s esatti di distanza dal termine dell'invio. (La RX2

è standardizzata in Europa su frequenza 869.525 MHz a DR0/SF12). Se il server vuole mandare un comando a un sensore in Classe A, deve accodarlo e aspettare il prossimo uplink spontaneo del nodo per infilarlo in una finestra RX.
2. **Classe B (Beacon)**: Risolve il limite di reperibilità della Classe A. Il Gateway invia "Beacon" regolari per sincronizzare l'orologio dei nodi. Grazie a questo riferimento, il nodo può aprire ulteriori "finestre di ricezione programmate" (es. ogni 10 minuti) in cui il server sa di poterlo raggiungere.
3. **Classe C (Continuous)**: Per i nodi perennemente collegati all'alimentazione elettrica (es. illuminazione pubblica, valvole industriali). La finestra di ricezione è sempre spalancata, la latenza di ricezione dei comandi è azzerata, ma il consumo è massimo.

#### 7.2.3 L'Ottimizzazione: ADR (Adaptive Data Rate)

Il meccanismo più brillante gestito dal Network Server. Il NS analizza storicamente gli SNR (Signal-to-Noise Ratio) degli ultimi pacchetti di un nodo in Classe A.

- Se il nodo è vicino (ottimo SNR) ma usa SF12, sta gridando lentamente sprecando batteria e occupando il canale radio per secondi inutilmente. Il NS manda un comando MAC (`LinkADRReq`) abbassando dinamicamente il suo SF (es. a SF7) e abbassando la potenza radio. Risultato: bit rate maggiore, frazione di secondo di Airtime, batteria salvata e rete decongestionata.
- L'ADR si occupa del compromesso vitale tra *Coverage, Airtime e Battery life*.

#### 7.2.4 Payload ed Efficienza: Da JSON a LPP

Inviare un JSON `{"pressure": 997.8, "temperature": 26.6, "humidity": 55.5}` consuma 54 byte (spesso letale o illegale ai duty cycle dell'SF12). Rinominalo in `{"p": 997, "t": 26, "h": 55}` scende a 30 byte, ma spreca spazio in parentesi e virgolette ASCII.
Si usa la codifica standardizzata **Cayenne LPP (Low Power Packet)**: comprime tutto in byte esadecimali posizionali. Trasforma gli stessi tre dati sensoriali completi in una stringa di soli **11 byte** (es. `00 73 26FA | 01 67 010A | 02 68 6F`, formattato in Canale|Tipo Sensore|Dato scalato grezzo).

#### 7.2.5 Modelli di Indirizzamento e Join Procedure (OTAA vs ABP)

Poiché LoRaWAN non è IP, usa ID hardware a 64-bit di base (`DevEUI`, `AppEUI/JoinEUI`) simili al MAC address. Per comunicare serve un Indirizzo di Rete temporaneo a 32-bit (`DevAddr`) e le chiavi AES-128 di sessione (`NwkSKey` per integrità della rete verso l'NS, `AppSKey` per decifrare il payload verso l'AS). L'attivazione (Join) ottiene questi dati.

1. **OTAA (Over-The-Air Activation)**: La procedura principe e dinamica.
   - Il nodo invia un generico *Join Request* all'etere.
   - Il Network Server autentica il nodo tramite un server centralizzato e genera freschissime *Session Keys* dinamiche in un *Join Accept*.
   - *Pro/Contro*: Altamente sicura, supporta la rotazione delle chiavi. Pienamente scalabile (migliaia di nodi gestiti centralmente). Richiede handshake bidirezionale inziale gravoso (consuma tempo e batteria). Se un nodo perde connessione deve ri-negoziare.
2. **ABP (Activation By Personalization)**: Procedura *hardcoded* "sorda e muta".
   - Il programmatore stampa a mano, bruciandoli nel firmware del dispositivo, il `DevAddr`, la `NwkSKey` e l'`AppSKey`.
   - *Pro/Contro*: Attivazione letteralmente istantanea e a zero energia/overhead di scambio (il nodo si accende e urla subito i dati). Svantaggi catastrofici in produzione: le chiavi sono "statiche a vita". Un compromesso fisico del device ruba i segreti permanenti, non scalabile per gestioni massificate. Valido solo per prototipi o sensori *extreme-low-power*.

---

### CAPITOLO 8: Routing nelle LLN e Protocollo RPL

Nelle Low-power and Lossy Networks (LLN) dell'IoT, la comunicazione è precaria, asimmetrica e imprevedibile. I protocolli *Distance Vector* tradizionali del mondo IP cablato (es. RIP, che usa il numero di salti o "hop count" come metrica, o OSPF, che inonda la rete con l'LSA) falliscono miseramente, producendo *black holes*, un altissimo *overhead* e loop infiniti ("count to infinity").

#### 8.1 La Metrica Assoluta: ETX (Expected Transmission Count)

Al MIT hanno rivoluzionato il routing wireless dimostrando che l'*hop count* è una metrica tossica (un solo lunghissimo hop radio al limite del campo, pur sembrando "1 salto", perderà il 99% dei pacchetti).
La metrica perfetta è l'**ETX**: "Il numero teorico probabile di trasmissioni necessarie per far sì che il pacchetto arrivi sano a destinazione includendo anche la corretta ricezione dell'ACK".

- *Il Modello Probabilistico*:
  $ETX = \frac{1}{P_{forward} \cdot P_{reverse}} = \frac{1}{1-p}$
- Un ETX di **1** indica un cavo ethernet puro (successo 100% al primo colpo).
- Se su 1000 pacchetti inviati e ritrasmessi il server ne segna 500 validi, l'ETX è $2$. Se per mandare 1024 pacchetti utili occorrono in totale 1898 trasmissioni, l'ETX sul link è $\approx 1.85$.
- È uno scalare additivo. Il costo di un intero percorso (*Path cost*) è la banale somma degli ETX dei vari link attraversati.

#### 8.2 RPL: Il Re dei Protocolli IPv6 WSN

L'IETF (gruppo ROLL) ha standardizzato il protocollo **RPL** (Routing Protocol for LLNs). RPL abbandona i vecchi schemi e crea algebricamente una struttura logica di rete chiamata **DODAG (Destination Oriented Directed Acyclic Graph)**, ovvero un grafo direzionato senza loop circolari dove ogni ramo punta inesorabilmente a scivolare giù verso un'unica destinazione centrale (la **Radice o Sink/Root**).

##### L'Architettura del DODAG

- **LBR (LLN Border Router)**: Solitamente fa da Root del DODAG. Fa da ponte verso la rete esterna.
- **Grounded vs Floating**: Un DODAG si dice *Grounded* se è ancorato saldamente a internet (attraverso un Root gateway forte). Se il link esterno crolla, o durante fasi di frammentazione della rete, segmenti isolati formeranno un DODAG *Floating* locale, che cercherà poi di riattaccarsi in massa al ramo principale.

- **Objective Function (OF)**: L'RPL è modulare. L'algoritmo matematico che definisce *come* valutare la "migliore" rotta è separato dal protocollo ed è chiamato *Objective Function*.
  - Esempi: **OF0** (minimizza brutalmente la latenza/hop), **MRHOF** (Minimum Rank with Hysteresis OF, sceglie il genitore basandosi sull'ETX globale per favorire i link radio limpidi). Le OF supportano anche vincoli negativi (*Constraint-based routing*), es. "escludi tutti i nodi a batteria e cerca percorsi solo su quelli attaccati a rete elettrica".
- **Il Rank**: È la grandezza numerica del DODAG. Non è l'Hop count, è la profondità/distanza dalla Root calcolata elaborando l'ETX attraverso l'OF.
  - La "Regola d'oro del Loop Avoidance": Un pacchetto può viaggiare "in su" solo verso un nodo con un **Rank numericamente inferiore** al proprio.
  - Nodi con Rank identico si chiamano *Siblings* (Fratelli).

##### I Messaggi di Controllo (Codificati come nuovi pacchetti ICMPv6)

RPL genera tutto usando tre nuovi messaggi ICMPv6, emessi in multicast:

1. **DIO (DODAG Information Object)**: Il messaggio "Top-Down". È l'annuncio emesso dalla Root verso l'esterno per dire "Sono qui, ecco le mie regole". I nodi vicini lo ricevono, calcolano il loro Rank in base alla OF specificata nel DIO, scelgono il mittente come loro **Parent** (genitore ottimale) e a loro volta rigenerano un DIO aggiungendoci il proprio Rank aumentato, propagando la struttura come un'onda. Trasporta parametri vitali come: DODAGID, DODAGSequenceNumber, RPLInstanceID (permette a una singola rete di far convivere 2 grafi sovrapposti per scopi di QoS diversi, es. uno per latenza, uno per affidabilità).
2. **DAO (Destination Advertisement Object)**: Il messaggio "Bottom-Up". Una volta costruito l'albero per far salire i dati, come fa il server a inviare un comando *in giù* verso uno specifico sensore terminale? I nodi "foglia" emettono DAO in salita per informare l'albero della loro raggiungibilità (i "prefissi IP" che possiedono). L'informazione si accumula (Reverse Route Stack o tabelle logiche in ogni antenato) in modo che i router conoscano le vie discese.
3. **DIS (DODAG Information Solicitation)**: Il grido d'aiuto. Un sensore appena sballato si accende. Invece di aspettare l'onda passiva di un DIO per ore, spara un DIS in broadcast: "C'è qualcuno?". I vicini accelerano forzatamente l'emissione del loro DIO permettendo l'aggancio.

##### La Gestione delle Direzioni del Traffico in RPL

1. **MP2P (Multipoint-to-Point / Uplink)**: Migliaia di sensori convergono dati al Sink. È la modalità nativa di un DODAG. Basta imboccare passivamente la *Default Route* verso il proprio Parent a rank inferiore.
2. **P2MP (Point-to-Multipoint / Downlink)**: Dal Sink verso uno specifico nodo (Attuazione). Sfrutta le informazioni DAO. RPL gestisce due approcci:
   - **Storing Mode**: Ogni nodo intermedio dell'albero memorizza (se ha RAM) la tabella di routing completa di tutti i nodi sottostanti ("foglie discendenti").
   - **Non-Storing Mode**: Nei dispositivi poveri. Nessuno salva tabelle in RAM. Il DAO viaggia interamente fino al Border Router portandosi dietro lo "storico dei balzi" (Reverse Route Stack). Quando il Sink deve rispondere, applica il **Source Routing IPv6**, "stampando" l'intero elenco degli IP da attraversare pezzo per pezzo come intestazione forzata sul pacchetto. Gli snodi in mezzo devono solo obbedire senza consultare tabelle.
3. **P2P (Point-to-Point)**: Da Nodo X a Nodo Y. Il pacchetto è costretto a viaggiare *in salita* seguendo l'albero verso la radice. Non appena incrocia un "Common Ancestor" (Antenato comune) che conosce la rotta "in discesa" verso Y, il pacchetto svolta giù, risparmiando di viaggiare inutilmente fino al Sink finale.

##### Ottimizzazione del Traffico di Controllo: Il Trickle Algorithm

Nelle reti instabili WSN, i protocolli storici emettevano messaggi di *hello/update* ogni $X$ secondi (overhead gigantesco). L'RPL fa magie col **Trickle Algorithm** applicato all'invio dei DIO.

- *Funzionamento*: Usa timer dinamici in base allo "stato" della rete. Se nulla cambia, il timer di inter-invio $I$ **raddoppia esponenzialmente** a ogni ciclo (da 2ms, a 4ms, 8, 16... fino ad arrivare a inviare un DIO ogni due ore). Se però un nodo sente un DIO di un nuovo vicino, o riceve un pacchetto non congruo (DODAGSequenceNumber incoerente, indice di topologia saltata), si scatena una "Inconsistency".
- *Risultato*: Il Trickle azzera il timer all'istante ($I = I_{min}$). La rete viene inondata di nuovi DIO a raffica, la topologia si ripara o si aggiorna a fulmine (pochi millisecondi). Una volta convergente e placata, il timer ricomincia a raddoppiare e in pochi minuti il network torna nel silenzio radio più assoluto, salvando le batterie.

##### Loop Avoidance e Autocura della Rete

Se il "Miglior Parent" (es. il Link verso il router) crolla:

1. **Riparazione Locale (Local Repair)**: Il nodo ferito analizza i vicini in base ai propri DAO. Sceglierà come genitore alternativo (Backup Parent) preferibilmente un fratello (Sibling di pari Rank). Se costretto sceglierà un nodo "sotto" di lui e scalerà dolorosamente incrementando numericamente il proprio Rank, pur di riconnettersi all'albero, ma garantendo che non vi sia inversione di rotta circolare verso la radice.
2. Se i parent falliscono irrimediabilmente, il nodo si taglia fuori dall'albero, trasformandosi in una sotto-radice indipendente (DAG Floating). Trascina i propri "figli" purificando le vecchie rotte (eliminazione per avvelenamento "poison reverse"). Quando la struttura floating intercetta un pacchetto DIO sano, si ri-aggancia al DODAG Grounded azzerando il problema. I timer *DAG Hop* e il DIO Sequence Number gestiscono questa sincronizzazione chirurgica a gradini.

---

#### CAPITOLO 9: Cybersecurity negli Ambienti IoT (Internet of Vulnerabilities)

In passato la security informatica si focalizzava su reti tradizionali (Enterprise). Nell'IoT i danni non sono virtuali, ma fisici. La cyber-Security evolve nella Physical-Safety. Se comprometti sensori *IoMT* (Sanità, monitor pazienti/robotici chirurgici), *Smart Cities* (traffico) o *IIoT* (reti IT e reti industriali OT SCADA interconnesse e vulnerabili), si sfocia in rischi vitali, operativi o sversamento chimico/energetico (Smart Grids).

##### 9.1 La Maledizione Intrinseca dell'IoT

1. I dispositivi spesso operano "Unattended" (Non presidiati) posizionati su pali in strada, sepolti nell'industria. Totalmente esposti alla manomissione hardware.
2. Soffrono la **Resource Exhaustion** (esaurimento risorse): cpu asfittica e batterie limitate implicano che cifrare costa tempo, RAM e preziosa energia in trasmissione.
3. Il firmware non testato ed eterogeneo diventa non-patchabile con cicli di vita di obsolescenza non previsti.

##### 9.2 Vettori e Domini di Attacco Estesi

1. **Dominio Fisico e Hardware**:

- Accesso alla scheda: L'attaccante dissalda la porta seriale/debug, fa il dump del firmware. Estrazione di *chiavi simmetriche di gruppo* da memoria non protetta (niente *tamper-resistance* chip TPM). Compromettendo la chiave di rete da un singolo sensore in campagna (clonazione device-key), compromette l'intero network a stella.
- *Firmware Replacement*: Upgrade del software tramite flash maligno, o introduzione nella supply chain di oggetti fisici manomessi in produzione (*hardware backdoor*).

2. **Dominio Rete e Routing**:
   - *Eavesdropping / MitM*: Ascolto intercettivo via radio o sniffaggio man-in-the-middle delle chiavi operative pre-condivise inviate in chiaro per errori di programmazione.
   - **Sinkhole (Blackhole) Attack**: Un nodo inietta messaggi di routing (RPL DIO) falsificati, dichiarandosi con il Rank minimo o il path migliore di tutta la rete (es. una finta stazione base vicina), attraendo gravitazionalmente tutto il flusso IoT circostante solo per ingoiarlo, bloccarlo, snasarlo o manometterlo.
   - **Sybil Attack**: Un singolo sensore malevolo invia pacchetti identificandosi ogni volta con un IP e MAC differente, ingannando le decisioni collaborative dei protocolli (es. bloccando i voti della rete).
   - **Selective Forwarding**: Peggio del blackhole, il nodo malevolo scarta chirurgicamente solo *alcuni* pacchetti specifici (es. gli allarmi antincendio) rendendo invisibile il disservizio alle sonde diagnostiche e al monitoraggio standard.
3. **Dominio Disponibilità (Denial-of-Service DoS)**:
   - *Jamming RF*: Emissione cieca di disturbi radio sulla frequenza della WSN bloccando la fisica T2T (Thing-to-Thing).
   - *Sleep Deprivation / Flooding*: L'attaccante tartassa il nodo sensore inviando finte richieste o ping asincroni, forzando la radio del dispositivo a risvegliarsi continuamente, esurendo a raffica in due giorni una batteria pensata per durare 10 anni (Battery draining). L'attaccante è spesso avvantaggiato da risorse illimitate via rete internet (dal backend).

#### 9.3 Contromisure Classiche e IPSec (Layer 3)

La sicurezza può essere garantita in base a *dove* si agisce, ma i protocolli tradizionali per Internet non sono ottimizzati per dispositivi *constrained*. Bisogna bilanciare crittografia simmetrica (AES) con quella asimmetrica (ECC, RSA - onerosissima in IoT per i calcoli e lo scambio chiavi).

La sicurezza implementata nel livello di rete (Layer 3) storicamente standardizzata è **IPSec** (RFC 4301). Nativo e un tempo obbligatorio nell'IPv6, opera in modo trasparente rispetto ai protocolli superiori implementando i servizi in due specifici protocolli EH (Extension Headers).

1. **AH (Authentication Header)**:
   - Assicura l'Integrità totale dei dati, l'Autenticazione esatta del mittente, e la protezione categorica anti-Replay.
   - *Funzionamento*: Applica una funzione hash con chiave (HMAC, ICV - Integrity Check Value). La particolarità dell'AH è che garantisce l'integrità anche sui campi sensibili, immutabili dell'intestazione IPv6 base (gli unici che ammette variare sono Hop limit ecc.).
   - *Campi Protocollo*: Identifica il payload che segue (Next Header), SPI (Security Parameter Index) a 32 bit per agganciare la *Security Association*, il Sequence Number (contatore tassativo a 32-bit per respingere i replay attacchi), ed in fondo lo spietato hash crittografico (ICV).
2. **ESP (Encapsulating Security Payload)**:
   - L'AH non cifra i dati in lettura. Se ti serve la Confidenzialità e il *secreto bancario*, usi ESP. Fornisce confidenzialità criptando completamente i payload o interi segmenti di traffico IP, e può garantire pure auth/integrità. Aggiunge un trailer (cifratura padding per arrotondare blocchi).

- **Le limitazioni di IPSec nell'IoT**: L'aggiunta in coda/testa dei byte strutturati degli header ESP e AH appesantisce il pacchetto IPv6 vanificando del tutto l'eleganza compressionale del 6LoWPAN HC/IPHC. Inoltre la gestione e lo scambio delle chiavi IKE (Internet Key Exchange) usa asimmetrica, un protocollo insostenibilmente grasso da caricare nella RAM e CPU di sensori primitivi che vivono con calcoli elementari. I nodi IoT spesso limitano e forzano all'uso IPSec pre-configurato via chiavi *Pre-Shared (PSK)* cablate offline.

#### 9.4 Protezione Livello Trasporto: TLS e la necessità di DTLS

**TLS (Transport Layer Security)**: La roccia sicura del web, posizionata ad albero sopra il TCP (L4). Protegge end-to-end con *Handshake protocol* (che include il check di certificati X.509, scambio asimmetrico delle chiavi segrete con RSA/Diffie-Hellman), per poi *Record protocol* (confidenzialità usando criptaggio a blocco simmetrico e check hash come SHA-1 contro intercettazione, alterazione e injection Man-in-the-Middle).
Ma l'IoT snello detesta TCP, usa UDP. TLS richiede tassativamente l'integrità del segmento di trasmissione e dei numeri di ordine affidabili, crolla immediatamente se un byte o un Record si inverte nel viaggio.
**DTLS (Datagram TLS)**: Nato per le WSN (e pilastro per COAP o MQTT-SN) è l'adattamento e l'innesto che rende la TLS amabile ai datagrammi instabili UDP.

- DTLS tollera brutalmente e assorbe l'inaffidabilità: include numerazioni di sequenza esplicita interne in modo che la *decifratura dei singoli record avvenga in totale indipendenza dai precedenti persi*. Gestisce i Timeout e implementa un ritrasmettitore autonomo di re-send qualora il pacchetto d'Handshake si sperda tra le nuvole.

#### 9.5 La Sfida Strutturale: Hop-By-Hop vs End-To-End (Sicurezza L7 e CBOR)

La problematica finale e irrisolta per i protocolli classici (IPSec, DTLS) emerge nelle vere reti IoT a maglia *Route-over / Mesh / Proxy-based*: questi protocolli in scenari multi-hop applicativi tutelano o solo il segmento radio punto-punto (*Hop-by-hop*) oppure falliscono l'E2E se un Proxy o IoT-Gateway intermedio funge da concentratore aggregatore dati o traduttore protocollo (Proxy CoAP/HTTP). A questi gateway intermedi è demandato dover "denudare" dal TLS e decrittare localmente l'header e il payload per reimpacchettarlo. L'E2E classico richiede una fiducia ceca nell'hardware di tutti i nodi del transito (trusted system) aprendo praterie d'intercettazione.

- **Sicurezza a Livello Applicazione**: La soluzione finale IoT è codificare robustezza non nel protocollo tubo di rete, ma nel payload stesso (Application-layer security). Si cripta, sigilla o firma solo l'informazione base. Esempi come *S/MIME, SRTP*, ecc. E qui sorge vitale la necessità della serializzazione microscopica.

#### 9.6 Il Disastro Ingegneristico dei Formati Classici: JSON vs CBOR (RFC 8949)

Cifrare dati comporta *padding*, blocchi AES, HMAC supplementari. E ricordiamoci, nella WSN un singolo pacchetto extra spacca un frame 802.15.4 in frammenti IP scatenando re-trasmissioni esiziali all'energia. I dati web standard JSON o XML in ASCII sono oscenità per l'IoT.

- **Esempio del "Grassone" JSON**: Un semplice payload sensore stringato: `{"temp":22, "humidity":48, "battery":2980}` consuma **40 byte** (il 49% dell'intero Payload teorico disponibile di un Frame fisico!).
- **CBOR (Concise Binary Object Representation)**: È un modello dati estensione binaria *Machine-efficient* strutturalmente mappabile da/per JSON, che non trasmette i caratteri "stringa" o parentesi in ASCII. Condensa le tipologie (Int, String, Array) in tag binari (es `A3` per Map3, `18 30` per un int unsigned). Il medesimo pacchetto Json in formato CBOR occupa solo **29 byte**.

- **Ottimizzazione estrema (CBOR Int-keys)**: Rimuovendo le stringhe *chiavi descrittive* verbose e mappandole su array a integer `1 → "temp", 2 → "humidity"` concordate con il backend, l'intero payload Json di 40 byte si rimpicciolisce a **10 miserissimi byte**. Efficienza assoluta. Sviluppata in tandem per questo formato binario c'è il sistema di sicurezza Applicativa parallela, battezzato **COSE (CBOR Object Signing and Encryption)**, che incarta direttamente il payload CBOR firmandolo asimmetricamente dentro la medesima codifica binaria senza sforare i limiti vitali della trasmissione Low-power radio.

---

**CAPITOLO 10: Sviluppi, Trend Tecnologici e Integrazioni Industriali Future**

Lo scenario tecnologico è uscito dalla sua infanzia di laboratorio (prototipi WSN) e si è cementato in distribuzioni *Carrier-grade*:

- **Maturità Tecnologica LPWAN**: LoRaWAN ha raggiunto lo stato dell'arte con oltre 125 milioni di dispositivi globali attivi (Smart Cities, Smart Metering per Multi-Utilities). Abbandonata l'era sperimentale (CAGR stabile del 25%).
- **L'Integrazione Satellitare NTN (Non-Terrestrial Networks)**: I limiti orizzontali di installazione Gateway terrestri vengono abbattuti dalla partnership con costellazioni LEO satellitari. Il satellite funge da backbone IP traslato, e sperimentale, il sensore LoRa comunicherà e griderà in uplink direttamente l'hardware orbitante sopra i deserti, aree rurali estreme e navigazione marittima. Si affronta una drastica e implacabile sfida fisica per superare limiti astronomici di latenza, orbita, asimmetria palese (downlink debolissimo o impossibile) e limitatezza vitale del Link Budget orbitale.
- **Evoluzioni Topologiche Multi-Banda e Spettro (LR-FHSS)**: Ristrutturazione protocolli, il sub-GHz continuerà ad agire sul long-range blindato, ma la scala enorme implementerà *Frequency-hopping espansi* (LR-FHSS) contro i jamming per consentire convivenza a migliaia di nodi senza la dipendenza mortale del CSS al duty cycle 1%. Abbinamenti del 2.4 GHz per l'High-throughput di prossimità.
- **Wi-Fi HaLow (802.11ah)**: Standard Wi-Fi radicalmente sub-GHz nato non per lottare sui consumi minimi dei bit di LoRa, ma per allargare a oltre 1 chilometro di distanza radio pura connettività IP e streaming standard. Taglia i ponti e i costi dei cavi industriali o Gateway LoraWAN offrendo alte performance intercompatibili all'ecosistema Router Wi-Fi esistente.
- **Traslazione Architetturale M2M / Decisionale OT**: Convergenza completa e traduttiva verso sistemi ed ecosistemi applicativi aziendali massivi preesistenti, portando la sensoristica WSN LoRa al livello della diagnostica integrata aziendale. **OPC UA (Open Platform Communications Unified Architecture)** diviene la mappa standard a cui agganciare, e instradare nativamente i dati raccolti "dal campo" verso le interfacce gestionali software e di controllo industriale SCADA (Supervisory Control And Data Acquisition) ed ERP. Lo Smart Object raccoglie con sensori ultra-basici l'etere, ma delega tutto il calcolo AI, previsionale di usura o predittivo, alle strutture Machine-Learning dei sistemi Cloud / Analytics Edge, divenendo così l'occhio cieco ma instancabile su macchinari ed eventi operativi. Il mercato WSN industriale sorpassa un valore globale di 4 Miliardi e frena la frammentazione proprietaria del decennio passato.